Multilingual Natural Language Inference over Swiss Voting Booklets

Project Information

- **Track:** Track 2A — Academia Challenges
- **Challenge:** OST — Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Team:** Craig D. Labs
- **Event:** Hack Apertus Online, October 2026
- **Status:** Functional Docker prototype verified with live Apertus inference; official held-out benchmark evaluation remains pending.

1. Summary

This project implements a document-grounded natural language inference (NLI) pipeline for Swiss official voting booklets. Given a claim and reference material, the system predicts entailment (0), neutral (1), or contradiction (2). It uses the Apertus-v1.5-8B model through the CSCS inference API and a local retrieval pipeline to select evidence passages from supplied reference text or extracted PDF content. The implementation supports German, French, and Italian input through multilingual text matching and evidence retrieval. Functional testing has verified direct-reference and PDF-backed processing with live Apertus inference inside Docker. The Makefile workflow has also been executed successfully, producing a prediction file with the expected output structure. These tests verify application functionality and inference integration; they do not establish official benchmark accuracy. A reproducible held-out evaluation remains outstanding.

2. Problem and Approach

The task is not general-purpose question answering. A claim must be judged only against the provided reference material. The system therefore aims to retrieve relevant passages before asking Apertus to classify the claim.

The pipeline consists of:

Read cases from JSON Lines input.

Load the supplied reference text or extract text from the specified PDF.

Segment the reference into candidate passages.

Rank passages using a hybrid lexical retrieval score.

Send a compact selection of passages and the claim to Apertus.

Parse the predicted label and return evidence and usage metrics.

The three labels are:

- `0`: entailment
- `1`: neutral
- `2`: contradiction

The implementation does not intentionally use outside knowledge to determine the relationship between a claim and its reference.

3. Architecture

Input handling

The command-line interface accepts an input JSONL file and an output JSONL file through `--input` and `--output`.

For direct-reference cases, the system reads the reference text included in the case. For PDF cases, it resolves the booklet path relative to the input file and extracts non-empty page text using `pypdf`.

Evidence retrieval

The current hybrid ranker combines four signals:

- Character trigram similarity
- Token similarity
- Exact anchors such as numbers, dates, percentages, and article references
- Heading overlap

The current weighted score is:

```
0.40 × character similarity + 0.25 × token similarity + 0.25 × anchor score + 0.10 × heading score
```

The current implementation selects up to three ranked passages for classification and returns up to five passages as evidence. The selected passages are intended to reduce context size while retaining information relevant to the claim.

These weights are an experimental baseline, not a proven optimum. Retrieval diagnostics showed a trade-off between larger passages, which can improve lexical overlap, and smaller passages, which reduce the amount of context sent to the model. The experiments do not establish a definitive winner across all languages and labels.

Classification

The classifier calls the Apertus chat-completions API with temperature set to zero and requests a single label. The model is configured using environment variables. The implementation supports both the challenge's documented `LLM_*` variable names and the existing CSCS-specific variable names.

Output

Each output record contains:

- `id`
- `label`
- `label_name`
- `evidence`
- `metrics`

The metrics record includes input tokens, output tokens, and inference time in milliseconds when supplied or measured by the implementation.

4. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B`
- **Use:** Inference for document-grounded NLI classification
- **Serving:** Hosted CSCS inference endpoint
- **Local model weights:** Not required
- **Prompting:** Explicit three-class label definitions; classify only against the supplied reference; do not use outside knowledge
- **Decoding:** Temperature 0; maximum output of five tokens

The model is accessed through an HTTP API. API credentials are supplied through environment variables and must not be committed to the repository.

5. Data

The development process used the OST multilingual NLI dataset, including the v1.1 data file, and selected Swiss official voting booklet PDFs for local retrieval diagnostics.

The dataset contains German, French, and Italian claims and references, including cross-lingual language pairs. The local development files are used for experimentation and smoke testing.

The data directory was measured at approximately 50.7 MiB during the latest development check, below the template's 100 MB limit. This measurement should be repeated against the exact final submission contents.

Before redistributing dataset files or booklet PDFs, confirm their applicable licences and redistribution conditions. Do not include API keys, private credentials, or other secrets in the repository.

6. Evaluation

Functional validation

The following local tests have passed:

- Python source compilation.
- Imports of the analyzer and CLI.
- CLI help invocation.
- Task B direct-reference smoke test using live Apertus inference inside Docker.
- Task A PDF-backed smoke test using live Apertus inference inside Docker.
- Output-schema validation for the Makefile run, including the expected prediction fields and five evidence items.
- Makefile workflow successfully built the Docker image, ran live inference, and wrote a validated prediction file.
- Git ignore check confirming that `.env` is ignored.

These tests verify the application pipeline and live inference integration on small smoke-test inputs. They do not measure official held-out benchmark performance.

Preliminary model and retrieval experiments

Development experiments included small diagnostic classification samples, cross-lingual retrieval comparisons, and comparisons of passage segmentation strategies. A preliminary 27-case language-pair diagnostic achieved 19/27 correct classifications (70.37%). This is a small development diagnostic, not an official held-out benchmark and not a reliable estimate of generalization performance.

Retrieval experiments compared the existing large-segment baseline with proposal-aware and semantic segmentation. The results showed a context-size versus lexical-overlap trade-off. Lexical overlap is an imperfect proxy for whether a passage contains sufficient evidence to support or contradict a claim, so these diagnostics are not presented as definitive retrieval accuracy.

Evaluation	Result	Interpretation
Task B live-inference smoke test	Passed	Docker execution verified on a direct-reference contract fixture
Task A live-inference smoke test	Passed	Docker execution verified on a PDF-backed example with page-referenced evidence
Makefile workflow	Passed	Image built, live inference completed, and prediction output validated
Preliminary 27-case diagnostic	19/27 (70.37%)	Small development sample only; not an official held-out benchmark
Official held-out evaluation	Not completed	Official benchmark performance has not been established
Docker image build and execution	Passed	Image built and executed successfully through Docker

A final submission should report a reproducible held-out macro-F1 result and the applicable evidence and efficiency metrics once those measurements have actually been obtained.

7. Limitations

- Retrieval relies on lexical and structural signals rather than a trained multilingual embedding model.
- Cross-lingual retrieval can fail when claim wording differs substantially from the reference language.
- PDF text extraction may introduce layout artefacts, split words, or lose structural information.
- A relevant passage may not be among the top-ranked segments.
- A correct passage does not guarantee a correct NLI label; numerical, conditional, and directional statements can still be misinterpreted.
- Neutral cases do not necessarily have a single supporting evidence passage.
- Smoke tests cover only a small number of examples and do not establish benchmark accuracy.
- Docker building, container execution, and the Makefile workflow have been verified locally, but full-dataset evaluation and official held-out performance remain unverified.
- Preliminary diagnostic samples are too small to establish robust performance across languages and labels.

8. Reproducibility

Requirements

- Python 3.11
- Dependencies listed in `requirements.txt`
- Access to the CSCS Apertus inference endpoint
- Valid API credentials supplied through environment variables
- Docker for the containerized execution expected by the challenge template
- GNU Make for the documented `make run` workflow

Environment variables

The implementation accepts the following documented names:

- `LLM_NAME`
- `LLM_BASE_URL`
- `LLM_API_KEY`

It also accepts the existing development names:

- `APERTUS_MODEL`
- `CSCS_API_BASE_URL`

- `CSCS_INFERENCE_API_KEY`

Do not commit the actual API key.

Local invocation

From the project root, run:

```
python -m src.cli --input data/contract_test.jsonl --output data/predictions.jsonl
```

This command performs live inference and requires valid credentials. It is shown for reproducibility; the contract fixture is a development test input, not the official evaluation dataset.

The Docker image and `make run` workflow have been verified locally. The command `make run OUTPUT=data/final_make_smoke_predictions.jsonl` built the image and completed live Apertus inference against the contract fixture. The output schema and five evidence items were validated. This confirms the container workflow for the smoke-test fixture; it is not a full-dataset benchmark result.

9. Next Steps

Verify the official evaluation input contract and run the final evaluator-compatible CLI.

Complete a reproducible held-out evaluation and record macro-F1 and efficiency metrics, if permitted evaluation data is available.

Inspect evidence quality on representative entailment and contradiction cases.

Recheck repository contents, data size, licences, and secret exclusion.

Record the final commit identifier and submission version.

Licensing and Attribution

The current Hack Apertus general terms specify Apache License 2.0 for source code, Creative Commons Attribution 4.0 (CC-BY-4.0) for documentation and other non-code materials, and Community Data License Agreement ? Permissive 2.0 (CDLA-Permissive-2.0) for submitted datasets, unless event-specific terms state otherwise.

These project-level licences do not automatically override third-party dataset or booklet licences. Before submission, verify the applicable licences and redistribution permissions for the OST dataset, the Swiss voting booklet PDFs, and all other included third-party material. Include the required attribution and licence information.

References

- Hack Apertus: https://hackapertus.ch/
- Online Hackathon: https://hackapertus.ch/online-hack
- OST challenge: https://hackapertus.notion.site/track-2a-ost
- OST dataset: https://huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets
- Apertus inference endpoint documentation: https://api.inference.cscs.ch/